Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторная работа №8 по курсу «Дискретный анализ»

Студент: Д. А. Широбоков
Преподаватель: С. А. Михайлова
Группа: М8О-301БВ
Дата:
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №8

Задача: Бычкам дают пищевые добавки, чтобы ускорить их рост. Каждая добавка содержит некоторые из N действующих веществ. Соотношения количеств веществ в добавках могут отличаться.

Воздействие добавки определяется как $c_1a_1 + c_2a_2 + \cdots + c_Na_N$, где a_i — количество i-го вещества в добавке, c_i — неизвестный коэффициент, связанный с веществом и не зависящий от добавки. Чтобы найти неизвестные коэффициенты c_i, Биолог может измерить воздействие любой добавки, использовав один её мешок.

Известна цена мешка каждой из M ($M \leq N$) различных добавок. Нужно помочь Биологу подобрать самый дешёвый набор добавок, позволяющий найти коэффициенты c_i. Возможно, соотношения веществ в добавках таковы, что определить коэффициенты нельзя.

Формат ввода

В первой строке записаны целые числа M и N. В каждой из следующих M строк записаны N чисел, задающих соотношение количеств веществ в добавке, а за ними — цена мешка добавки. Порядок веществ во всех описаниях добавок один и тот же. Все числа — неотрицательные целые, не превосходящие 50.

Формат вывода

Выведите -1, если определить коэффициенты невозможно. Иначе выведите набор добавок в виде их номеров (добавки нумеруются по порядку во входных данных). Если вариантов несколько, выведите любой из них.

1 Описание

Требуется выбрать самый дешёвый набор пищевых добавок, по измерениям воздействия которых можно однозначно найти неизвестные коэффициенты $c_1, \ldots, c_N$, либо сообщить, что это невозможно.

Каждая добавка описывается вектором $a = (a_1, \ldots, a_N)$ и ценой. Измерив воздействие добавки i, мы получаем линейное уравнение $c_1 a_{i1} + \cdots + c_N a_{iN} = y_i$. Для выбранного набора добавок получается система линейных уравнений относительно $c_1, \ldots, c_N$, и она имеет единственное решение тогда и только тогда, когда ранг матрицы из векторов выбранных добавок равен N. Значит, коэффициенты можно определить, только если ранг матрицы всех M добавок равен N, иначе нужно вывести -1.

Если ранг равен N, то среди добавок нужно найти набор минимальной стоимости, ранг которого равен N. Цены неотрицательны, поэтому из любого такого набора можно выбросить линейно зависимые добавки, не увеличив стоимость. Следовательно, оптимальный набор состоит из ровно N линейно независимых векторов, то есть является базисом.

Множество строк матрицы вместе с семейством линейно независимых подмножеств образует матроид, а для задачи о базисе минимального веса в матроиде жадный алгоритм даёт оптимальный ответ. Интуитивно это следует из того, что добавка, лежащая в линейной оболочке более дешёвых уже выбранных добавок, ничего не добавляет к рангу, и её можно заменить более дешёвыми. Поэтому алгоритм такой:

1. отсортировать добавки по возрастанию цены;
2. перебирать добавки в этом порядке и брать очередную, если она линейно независима от уже взятых;
3. остановиться, когда набрано N добавок. Если добавки закончились раньше, вывести -1.

Линейная независимость проверяется методом Гаусса. Для каждого столбца k хранится индекс строки `basis[k]`, у которой первый ненулевой элемент стоит в столбце k и равен единице. Очередная строка последовательно приводится по этим базисным строкам: для каждого столбца k, в котором элемент строки не равен нулю, либо вычитается базисная строка (если она есть), либо строка сама становится базисной строкой для столбца k после деления на этот элемент. Если строка свелась к нулю, она линейно зависима от выбранных и пропускается. Столбец цены в вычитании не участвует.

Так как цены — целые числа от 0 до 50, сортировка выполняется сортировкой подсчётом. Она устойчива и запоминает исходные номера добавок, которые нужны для вывода. Найденные номера затем тоже сортируются подсчётом по возрастанию.

Сложность по времени: сортировка $O(M + C)$, где $C = 50$ — число возможных цен, приведение одной строки $O(N^2)$ (не более N вычитаний по $O(N)$ каждое), всего $O(MN^2)$, а так как $M \leq N$, то не более $O(N^3)$. Сложность по памяти — $O(MN)$.

Вычисления ведутся в числах с плавающей точкой (`double`), ноль определяется сравнением с $\varepsilon = 10^{-10}$. Для небольших N и коэффициентов не больше 50 этого достаточно, но при больших N накопление ошибок округления может привести к неверному определению ранга. Точные варианты — вычисления в дробях или по модулю большого простого числа.

2 Исходный код

Исходные данные хранятся в матрице `input` размера $M \times (N+1)$: последний элемент каждой строки — цена. Функция `counting_sort_input` устойчиво сортирует строки по цене сортировкой подсчётом и возвращает массив `permutations`, где `permutations[j]` — исходный номер строки, оказавшейся на позиции j. Исходная матрица после сортировки освобождается.

Затем строки обрабатываются в порядке возрастания цены. Массив `basis` размера N хранит для каждого столбца номер базисной строки (или -1). Для очередной строки внутренний цикл идёт по столбцам: нулевые элементы пропускаются, при отсутствии базисной строки в столбце текущая строка становится базисной (нормируется делением на ведущий элемент), иначе из неё вычитается базисная строка. Внутренние циклы по j идут от $N-1$ к k, чтобы элемент k, на который умножаются и делятся остальные, изменялся последним. Переменная `full_basis` позволяет остановиться, когда базис уже полон.

После обработки собираются исходные номера базисных строк. Если их меньше N, выводится -1, иначе номера сортируются функцией `counting_sort_ans` и выводятся.

```
1  #include <algorithm>
2  #include <iostream>
3  #include <utility>
4  #include <vector>
5
6  #define MAX_PRICE 50
7  #define EPS 1e-10
8
9  std::vector<int> counting_sort_input(std::vector<std::vector<double>>& v, std::vector<
       std::vector<double>>& res) {
10     std::vector<int> count(MAX_PRICE + 1);
11     for (int i = 0; i < v.size(); ++i) {
12         ++count[v[i].back()];
13     }
14     for (int i = 1; i < count.size(); ++i) {
15         count[i] += count[i - 1];
16     }
17     std::vector<int> permutations(v.size());
18     res.resize(v.size());
19     for (int i = v.size() - 1; i >= 0; --i) {
20         int j = --count[v[i].back()];
21         res[j] = std::move(v[i]);
22         permutations[j] = i;
23     }
24     return permutations;
25 }
26
```

```
27 std::vector<int> counting_sort_ans(std::vector<int>& v) {
28     std::vector<int> count(1 + *std::max_element(v.begin(), v.end()));
29     for (int i = 0; i < v.size(); ++i) {
30         ++count[v[i]];
31     }
32     for (int i = 1; i < count.size(); ++i) {
33         count[i] += count[i - 1];
34     }
35     std::vector<int> res(v.size());
36     for (int i = v.size() - 1; i >= 0; --i) {
37         int j = --count[v[i]];
38         res[j] = v[i];
39     }
40     return res;
41 }
42 
43 int main(void) {
44     int m, n;
45     std::cin >> m >> n;
46 
47     std::vector<std::vector<double>> input;
48     input.resize(m);
49     for (int i = 0; i < m; ++i) {
50         input[i].resize(n + 1);
51         for (int j = 0; j <= n; ++j) {
52             std::cin >> input[i][j];
53         }
54     }
55 
56     std::vector<std::vector<double>> bads;
57     auto permutations = counting_sort_input(input, bads);
58     input.clear();
59 
60     std::vector<int> basis;
61     basis.resize(n, -1);
62 
63     for (int i = 0; i < m; ++i) {
64         bool full_basis = true;
65         for (int k = 0; k < n; ++k) {
66             full_basis &= (basis[k] != -1);
67 
68             if (std::abs(bads[i][k]) < EPS) {
69                 continue;
70             }
71 
72             if (basis[k] == -1) {
73                 basis[k] = i;
74                 for (int j = n-1; j >= k; --j) {
75                     bads[i][j] /= bads[i][k];
```

```
76                  }
77                  break;
78              }
79
80              for (int j = n-1; j >= k; --j) {
81                  bads[i][j] -= bads[i][k] * bads[basis[k]][j];
82              }
83          }
84
85          if (full_basis) {
86              break;
87          }
88      }
89
90      std::vector<int> ans;
91      for (int i = 0; i < n; ++i) {
92          if (basis[i] == -1) {
93              break;
94          }
95          ans.push_back(permutations[basis[i]] + 1);
96      }
97
98      if (ans.size() < n) {
99          std::cout << -1;
100     } else {
101         ans = counting_sort_ans(ans);
102         for (auto& i : ans) {
103             std::cout << i << " ";
104         }
105     }
106 }
```

3 Консоль

```
laba-8>cmake -S . -B build
laba-8>cmake --build build --config Release
laba-8>type t1
3 3
1 0 0 5
0 1 0 3
0 0 1 4
laba-8>build\Release\main.exe <t1
1 2 3
laba-8>type t2
3 2
```

```
1 1 2
1 0 5
0 1 1
laba-8>build\Release\main.exe <t2
1 3
laba-8>type t3
2 2
1 1 3
2 2 4
laba-8>build\Release\main.exe <t3
-1
```

Пояснения к тестам. В первом тесте все три вектора линейно независимы, поэтому нужны все добавки. Во втором тесте сначала берётся самая дешёвая добавка 3 (цена 1), затем добавка 1 (цена 2), которая независима от неё; дорогая добавка 2 (цена 5) уже не нужна, так как ранг равен 2. В третьем тесте добавка 2 пропорциональна добавке 1, ранг матрицы равен 1 при $N = 2$, поэтому коэффициенты определить нельзя.

4 Тест производительности

Тест производительности состоит из трёх частей. В первой результаты жадного алгоритма сравниваются с наивным перебором на случайных малых тестах, чтобы проверить корректность. Во второй время жадного алгоритма сравнивается со временем перебора всех подмножеств добавок при растущем M. В третьей время жадного алгоритма измеряется при росте N, чтобы проверить оценку $O(MN^2)$.

Наивный перебор проходит по всем подмножествам добавок, пропускает те, в которых меньше N добавок или стоимость не меньше лучшей найденной, а для остальных независимо вычисляет ранг матрицы методом Гаусса с выбором опорного элемента. Среди подмножеств ранга N выбирается самое дешёвое. Количество подмножеств равно 2^M, поэтому время перебора растёт экспоненциально.

Время измерялось с помощью `std::chrono::steady_clock`. Для каждого алгоритма замер проводился 3 раза, в таблицах указано среднее значение. Измерялось только решение (сортировка и метод Гаусса); чтение ввода и вывод не учитывались, а копирование матрицы входит в замер. Для замеров использовались случайные матрицы с коэффициентами и ценами от 0 до 50.

Корректность. Результаты жадного алгоритма сравнивались с перебором на 3000 случайных тестах (N от 1 до 5, M от N до $N + 5$, коэффициенты от 0 до 2, цены от 0 до 10). Малые коэффициенты и цены выбраны специально, чтобы получалось много линейно зависимых строк и одинаковых цен. В каждом тесте проверялось, что ответ -1 получается одновременно у обоих алгоритмов, что стоимость найденного набора равна оптимальной и что ранг выбранных добавок равен N. Все 3000 тестов пройдены, при этом в 222 из них ответом было -1.

Сравнение с перебором. Число веществ $N = 5$, число добавок M растёт.

M	N	Жадный алгоритм, с	Перебор, с	Отношение
8	5	0,000006880	0,000028133	4,1
12	5	0,000007280	0,000342533	47,1
16	5	0,000007060	0,004736333	670,9
20	5	0,000006400	0,089583433	13 997,4

Масштабирование по N. Число добавок $M = 2N$. Измерялись два сценария. В первом матрица имеет полный ранг, и алгоритм останавливается, как только набран базис. Во втором последний столбец матрицы нулевой, ранг равен $N - 1$, базис полным не становится, и алгоритм обрабатывает все M строк, то есть работает в худшем случае.

В столбце «Рост» указано отношение времени худшего случая ко времени для предыдущего N. В последнем столбце время худшего случая разделено на MN^2 и выражено в наносекундах.

N	M	Полный ранг, с	Ранг $N-1$, с	Рост	нс/(MN^2)
50	100	0,000136	0,000236	—	0,943
100	200	0,000845	0,001858	7,88	0,929
200	400	0,008233	0,014951	8,05	0,934
400	800	0,051325	0,118204	7,91	0,923
800	1600	0,429014	1,040611	8,80	1,016

```
PS laba-8>.\build\Release\bench.exe
Correctness vs brute force: OK (3000 random tests,222 of them with answer -1)

M    N    greedy,s        brute force,s   ratio
8    5    0.000006880      0.000028133      4.1
12   5    0.000007280      0.000342533      47.1
16   5    0.000007060      0.004736333      670.9
20   5    0.000006400      0.089583433      13997.4

N      M      full rank,s     rank N-1,s      growth     ns/(M*N^2)
50     100    0.000136         0.000236         -         0.943
100    200    0.000845         0.001858        x7.88      0.929
200    400    0.008233         0.014951        x8.05      0.934
400    800    0.051325         0.118204        x7.91      0.923
800    1600   0.429014         1.040611        x8.80      1.016
```

Из первой таблицы видно, что время перебора растёт экспоненциально: каждое увеличение M на 4 умножает его в 12–19 раз, что соответствует росту 2^M (при увеличении M на 4 число подмножеств вырастает в 16 раз). При $M = 20$ перебор уже работает около 0,09 с и в 14 тысяч раз медленнее жадного алгоритма, а при больших M он станет неприменим. Время жадного алгоритма на таких малых данных остаётся на уровне нескольких микросекунд и от M почти не зависит: оно определяется накладными расходами (выделение памяти, копирование матрицы), поэтому небольшие колебания не отражают реальной асимптотики.

Вторая таблица подтверждает оценку $O(MN^2)$. Так как $M = 2N$, время должно расти пропорционально N^3, то есть при удвоении N примерно в 8 раз, что и наблюдается (отношения 7,9–8,8). Время, приходящееся на единицу MN^2, остаётся в пределах 0,92–1,02 нс, то есть практически постоянно. Небольшое увеличение при $N = 800$ (рост в 8,8 раза, 1,016 нс) вероятно связано с кэшем процессора: матрица 1600×801 чисел типа `double` занимает около 10 МБ и не помещается в кэш, поэтому обращения к памяти становятся медленнее. На асимптотику это не влияет. В худшем случае ($N = 800$, $M = 1600$) решение работает около одной секунды.

Сценарий с полным рангом быстрее худшего случая в 1,7–2,4 раза, потому что алго-

ритм прекращает работу после набора базиса и не обрабатывает оставшиеся строки. Он также растёт примерно в 8 раз при удвоении N.

5 Выводы

Выполнив восьмую лабораторную работу по курсу «Дискретный анализ», я познакомился с жадными алгоритмами и применил их к задаче о выборе самого дешёвого набора пищевых добавок, по измерениям которых можно найти неизвестные коэффициенты.

Главное, что я понял: жадный алгоритм работает не везде, а только когда у задачи есть подходящая структура. Здесь она появляется после сведения задачи к линейной алгебре. Коэффициенты можно определить тогда и только тогда, когда ранг матрицы выбранных добавок равен числу веществ N. Тогда искомый набор является базисом минимальной стоимости, а множества линейно независимых строк образуют матроид, для которого жадный выбор по возрастанию цены оптимален [1]. Без этого наблюдения жадную сортировку по цене нельзя было бы обосновать, и мне пришлось бы перебирать подмножества.

На практике я научился проверять линейную независимость строки относительно уже выбранных методом Гаусса, храня для каждого столбца индекс базисной строки. Здесь важен порядок обработки: внутренние циклы идут от последнего столбца к текущему, чтобы элемент, на который умножаются и делятся остальные, изменялся последним. Ещё одна техническая деталь — сортировка подсчётом: она устойчива, работает за $O(M + C)$ при ограниченных ценах и позволяет сохранить исходные номера добавок, нужные для вывода.

Слабое место решения — вычисления в числах с плавающей точкой: ноль определяется сравнением с $\varepsilon = 10^{-10}$, поэтому при больших N ошибки округления могут исказить определение ранга. Для строгой гарантии пришлось бы использовать вычисления в дробях или по модулю большого простого числа. Кроме того, тесты на больших N использовали случайные данные, поэтому я не могу утверждать, что на специально подобранных тестах погрешности не проявятся.

Список литературы

[1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 2-е издание.* — Издательский дом «Вильямс», 2007. Перевод с английского: И. В. Красиков, Н. А. Орехова, В. Н. Романов. — 1296 с. (ISBN 5-8459-0857-4 (рус.))

[2] *Жадный алгоритм — Википедия.*
URL: `https://ru.wikipedia.org/wiki/Жадный_алгоритм` (дата обращения: 27.10.2026).

[3] *Метод Гаусса — Википедия.*
URL: `https://ru.wikipedia.org/wiki/Метод_Гаусса` (дата обращения: 27.10.2026).